

Méthodes d'intégration à pas adaptatif

- But : résoudre une équation différentielle linéaire ou non-linéaire

$$\frac{dy}{dt} = f(t, y) \quad \text{tel que} \quad y(t_0) = y_0$$

- Ce sont des techniques plus versatiles gérant elle-même le choix du pas de temps d'intégration.
- En premier, une valeur acceptable pour l'erreur d'intégration est donnée par l'utilisateur. On la note ε_m
- A chaque itération, le schéma tente d'ajuster le pas d'intégration de manière à ce que l'erreur ε ne dépasse pas la valeur de l'erreur acceptable ε_m .

Principe de ces techniques

- Supposons que le vecteur $\mathbf{y}(t_n)$ soit connu à l' instant t_n
- L' idée est d' effectuer l' intégration du système d' équations différentielles de l' instant t_n à l' instant t_n+h , **deux** fois avec **deux** algorithmes différents .
- Chacun donne une estimation de $\mathbf{y}(t_n+h)$ que l' on note A_1 et A_2
- On utilise A_1 et A_2 pour calculer une estimation de l' erreur commise lors de l' intégration.
- En combinant linéairement A_1 et A_2 on augmente d' un ordre en h la précision sur $\mathbf{y}(t_n+h)$

Méthode d' Euler et d' Euler à 2 pas

- On note $\phi(t)$ la solution exacte de l' éq. diff. $\frac{dy}{dt} = f(t, y)$ tq $y(t_0) = y_0$
- Sans perdre en généralité, on suppose qu' à l' instant n , $\phi(t_n) = y_n$

Phase 1 : application de la méthode d' Euler 1x avec pas= h donne :

$$A_1 = y_n + h f(t_n, y_n)$$

Or

$$\phi(t_n + h) = \phi(t_n) + h \phi'(t_n) + \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

$$= y_n + h f(t_n, y_n) + \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

$$A_1 = \phi(t_n + h) - \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

Erreur commise lors de l' intégration avec Euler

Phase 2 : Application de la méthode RK2 avec un pas=h

$$\begin{cases} y_{mid} = y_n + \frac{h}{2} f(t_n, y_n) \\ A_2 = y_{mid} + h f(t_n + \frac{h}{2}, y_{mid}) \end{cases} \Rightarrow A_2 = y_n + h f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right)$$

$$\begin{aligned} f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right) &= f(t_n, y_n) + \frac{h}{2} f(t_n, y_n) \frac{\partial f}{\partial y}(t_n, y_n) + \frac{h}{2} \frac{\partial f}{\partial t}(t_n, y_n) + \mathcal{O}(h^2) \\ &= f(t_n, y_n) + \frac{h}{2} \left(f(t_n, y_n) \frac{\partial f}{\partial y}(t_n, y_n) + \frac{\partial f}{\partial t}(t_n, y_n) \right) + \mathcal{O}(h^2) \\ &= f(t_n, y_n) + \frac{h}{2} \left(\frac{df}{dt}(t_n, y_n) \right) + \mathcal{O}(h^2) \\ &= f(t_n, y_n) + \frac{h}{2} (\phi''(t_n)) + \mathcal{O}(h^2) \end{aligned}$$

car $\phi(t_n) = y_n$ et $\frac{dy}{dt} = f(t, y)$

D'où :
$$A_2 = y_n + \frac{h}{2} \left(f(t_n, y_n) + \frac{h}{2} (\phi''(t_n)) \right) = y_n + h f(t_n, y_n) + \frac{h^2}{2} (\phi''(t_n))$$

$$A_2 = y_n + h f(t_n, y_n) + \frac{h^2}{2} \phi''(t_n)$$

que l'on peut écrire

$$A_2 = y_n + h \phi'(t_n) + \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$



$$A_2 = \phi(t_n + h) + \mathcal{O}(h^3)$$

Comme $A_1 = \phi(t_n + h) - \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$

l'erreur d'intégration ε à chaque pas peut être estimée par $A_1 - A_2$

$$A_1 - A_2 = -\frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

$$\varepsilon = |A_1 - A_2|$$

A chaque pas, l'erreur ε est calculée

Algorithme de principe

Si $\varepsilon > \varepsilon_m$ alors

l'itération est rejetée

On diminue le pas de temps selon

$$h' = \sqrt{\frac{\varepsilon_m}{\varepsilon}}.h$$

$$A_1 = \phi(t_n + h) - \frac{h^2}{2} \phi''(t_n) + \mathcal{O}(h^3)$$

Sinon

l'itération est validée

$$A_2 = \phi(t_n + h) + \mathcal{O}(h^3)$$

$$y_{n+1} = A_2 + \mathcal{O}(h^3)$$


On augmente le pas de temps selon

$$h' = \sqrt{\frac{\varepsilon_m}{\varepsilon}}.h$$

Finsi

Rem : en pratique, un facteur de sécurité est appliqué à l'expression de h'

$$h' = 0.9 \sqrt{\frac{\varepsilon_m}{\varepsilon}}.h$$

Méthode de Kutta -Merson

$$\left\{ \begin{array}{l} K_1 = h f(y_n, t_n) \\ K_2 = h f(t_n + \frac{h}{3}, y_n + \frac{K_1}{3}) \\ K_3 = h f(t_n + \frac{h}{3}, y_n + \frac{K_1}{6} + \frac{K_2}{6}) \\ K_4 = h f(t_n + \frac{h}{2}, y_n + \frac{K_1}{8} + \frac{3K_3}{8}) \\ K_5 = h f(t_n + h, y_n + \frac{K_1}{2} - \frac{3K_3}{2} + 2K_4) \end{array} \right. \quad \text{et} \quad \left\{ \begin{array}{l} A_1 = y_n + \frac{K_1}{2} - \frac{3K_3}{2} + 2K_4 \\ \quad = \phi(t_n + h) - \frac{h^5}{120} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \\ A_2 = y_n + \frac{K_1}{6} + \frac{2K_4}{3} + \frac{K_5}{6} \\ \quad = \phi(t_n + h) - \frac{h^5}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \end{array} \right.$$

C' est une méthode d'intégration d'ordre 5

L'erreur d'intégration à chaque pas peut être estimée



$$A_1 - A_2 = -\frac{5h^5}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6)$$

$$\varepsilon = \frac{-h^5}{720} \phi^{(5)}(t_n) = \frac{1}{5} (A_1 - A_2)$$

A chaque pas, l'erreur $\varepsilon = \frac{1}{5}(A_1 - A_2)$ est calculée

Si $\varepsilon > \varepsilon_m$ alors

l'itération est rejetée

On diminue le pas de temps selon

$$h' = \left(\frac{\varepsilon_m}{\varepsilon} \right)^{1/5} . h$$

Sinon

l'itération est validée

$$y_{n+1} = A_2 - \varepsilon + \mathcal{O}(h^6)$$

$$\begin{cases} A_1 = \phi(t_n + h) - \frac{h^5}{120} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \\ A_2 = \phi(t_n + h) - \frac{h^5}{720} \phi^{(5)}(t_n) + \mathcal{O}(h^6) \end{cases}$$

On augmente le pas de temps selon

$$h' = \left(\frac{\varepsilon_m}{\varepsilon} \right)^{1/5} . h$$

Finsi

Rem : en pratique, un facteur de sécurité est appliqué à l'expression de h'

$$h' = 0.9 \left(\frac{\varepsilon_m}{\varepsilon} \right)^{1/5} . h$$

ODE23 et ODE45

ode23 et ode45 sont des solveurs implantés dans Matlab®

ode23 : Méthode de Runge-Kutta-Fehlberg 23

utilisation des méthodes de Runge-Kutta d'ordres 2 et 3

ode45 : Méthode de Runge-Kutta-Fehlberg 45 ~ Kutta - Merson

utilisation des méthodes de Runge-Kutta d'ordres 4 et 5

Utilisation d' ODE45

$[t,Y]=ode45 (@f, [t_0 \ t_{fin}],y_0)$

f est le nom de la fonction où est défini le système d' EDO à résoudre

t0 et tfin sont les valeurs initiale et finale de la variable d' intégration (par ex. le temps)

y0 est le vecteur ligne contenant les conditions initiales

Y est une matrice qui contient le résultat de l' intégration; chaque colonne correspond à une variable et chaque ligne correspond à une valeur de la variable indépendante (vecteur **t**)

Matlab décide par défaut de prendre un pas de temps initial d' intégration de $(t_{fin}-t_0)/1000$.

Utiliser `odeset` pour passer des paramètres à l' intégrateur

```
opts = odeset('MaxOrder', 2, 'InitialStep',1e-9, 'MaxStep',1e-8, 'stats','off')
newopts = odeset(opts);
```

Exemple :ODE45 pour l'oscillateur harmonique

```
function solver_ode45
```

```
hold off
```

```
tfin=12;
```

```
x0=1;    v0=0;
```

```
[T,Y] = ode45(@oscil,[0 tfin],[x0 v0]);
```

```
plot(T,Y(:,1),'-',T,Y(:,2),'-.')
```

```
save 'mydata.mat'
```

pas initial d'intégration (tfin-t0)/1000

Modification des paramètres par défaut des intégrateurs comme ode45 avec la commande odeset

```
function dy = oscil(t,y)
```

```
dy = zeros(2,1);
```

```
% ATTENTION vecteur ligne --> colonne
```

```
% pour compatibilite avec ode45 de MATLAB
```

```
%2eme cas
```

```
n=0.2;  p2=1;
```

```
% changement de variable pour garder les notations de l'équation
```

```
x=y(1); v=y(2);
```

```
% écriture de l'équation telle quelle
```

```
dx_dt = v;
```

```
dv_dt = -p2*x-2*n*v;
```

```
% changement de variable pour se conformer au vecteur de sortie dy
```

```
dy(1)=dx_dt;
```

```
dy(2)=dv_dt;
```